

Σχεδιασμός Ολοκληρωμένων Κυκλωμάτων II

Booth multiplier 8-bit project

Ομάδα 7

Βασίλειος Βλάχος (AM: 1100832)

Βιολέτα Μέγαρη (AM: 1100938)

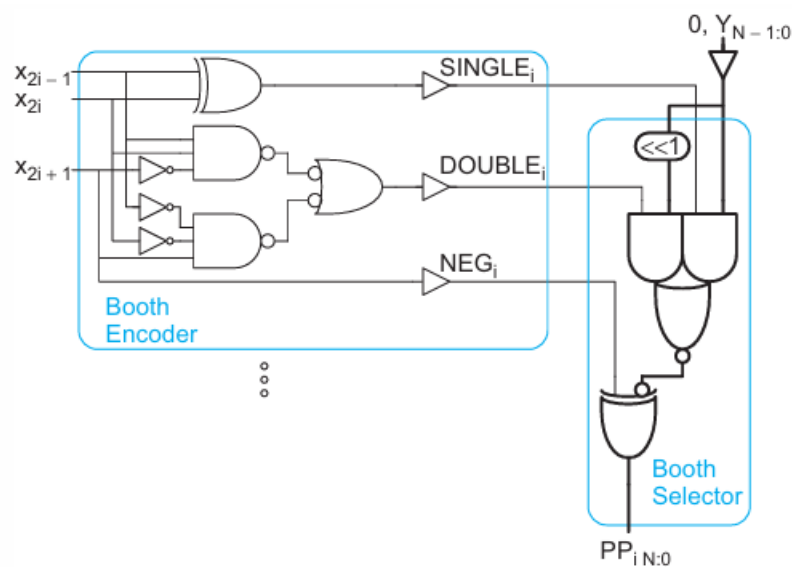
Στην συγκεκριμένη εργασία υλοποιείται σε vhdl ένας radix-4 πολλαπλασιαστής Booth των 8 bits και στην συνέχεια γίνεται σύνθεση και σχεδίαση του layout του με χρήση genus και innovus.

Ο πολλαπλασιαστής Booth πραγματοποιεί τον κλασικό πολλαπλασιασμό δύο όρων, δηλαδή δημιουργία των αντίστοιχων μερικών γινομένων και άθροισή τους, αφού πρώτα κωδικοποιήσει με χρήση Booth encoding τον έναν από αυτούς τους όρους. Ένας τυπικός radix-2 πολλαπλασιαστής απαιτεί την δημιουργία και την άθροιση 8 μερικών γινομένων, ενώ η χρήση radix-4 μειώνει αυτά τα μερικά γινόμενα σε 4. Σε αυτή την περίπτωση όμως απαιτείται ο υπολογισμός «hard multiples» ενός όρου, όπως για παράδειγμα το γινόμενο $3Y$, όπου Y είναι ένας όρος των 8 bits. Το γινόμενο αυτό προκύπτει από την άθροιση του Y με το $2Y$, το οποίο χρειάζεται έναν αργό carry propagate adder (CPA) για να υπολογιστεί. Η βασική ιδέα της κωδικοποίησης Booth είναι η χρήση ολισθήσεων και αφαιρέσεων ενός όρου ώστε να παραχθούν τα πολλαπλάσιά του. Για παράδειγμα, ο όρος $3Y$ παράγεται εκτελώντας την πράξη $4Y-Y$, το οποίο πρακτικά σημαίνει ότι το πρώτο μερικό γινόμενο θα είναι ίσο με $-Y$ (δηλαδή Y σε συμπλήρωμα ως προς 2), ενώ στο δεύτερο μερικό γινόμενο (που λόγω του radix-4 έχει βάρος 2) θα προστεθεί ένα επιπλέον Y κι έτσι αποφεύγεται η χρήση του αργού CPA.

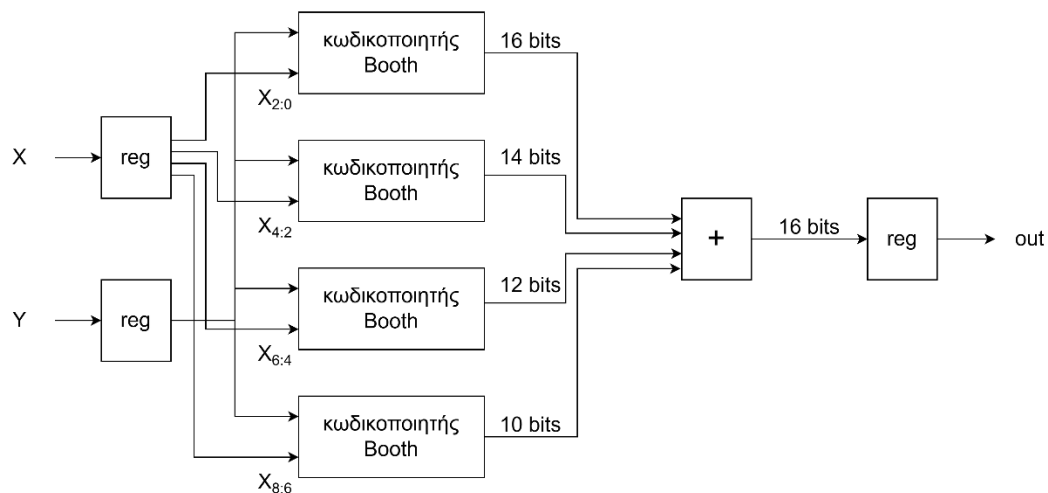
Θεωρώντας τον radix-4 πολλαπλασιασμό δύο 8-bit όρων $P=Y*X$, όπου $Y=y_7y_6\dots y_1y_0$ και $X=x_7x_6\dots x_1x_0$ με τους δείκτες να αντιστοιχούν στο βάρος κάθε ψηφίου, ο πολλαπλασιαστής Booth δουλεύει ως εξής: Αρχικά προστίθεται ένα μηδενικό bit στο τέλος του X (στην θέση με βάρος -1) και χρησιμοποιείται η τριάδα ψηφίων $x_1x_0x_{-1}$ για την εξαγωγή του κατάλληλου μερικού γινομένου. Το επόμενο μερικό γινόμενο προκύπτει από την τριάδα $x_3x_2x_1$ κλπ. Τα μερικά γινόμενα προκύπτουν από τις τιμές των τριών ψηφίων σύμφωνα με τον ακόλουθο πίνακα αληθείας [1].

Inputs			Partial Product	Booth Selects		
x_{2i+1}	x_{2i}	x_{2i-1}	PP_i	$SINGLE_i$	$DOUBLE_i$	NEG_i
0	0	0	0	0	0	0
0	0	1	Y	1	0	0
0	1	0	Y	1	0	0
0	1	1	$2Y$	0	1	0
1	0	0	$-2Y$	0	1	1
1	0	1	$-Y$	1	0	1
1	1	0	$-Y$	1	0	1
1	1	1	$-0 (= 0)$	0	0	1

Με βάση αυτόν τον πίνακα προκύπτει το διάγραμμα σε επίπεδο πυλών του κωδικοποιητή Booth που παράγει κάθε μερικό γινόμενο, όπως φαίνεται στην παρακάτω εικόνα.



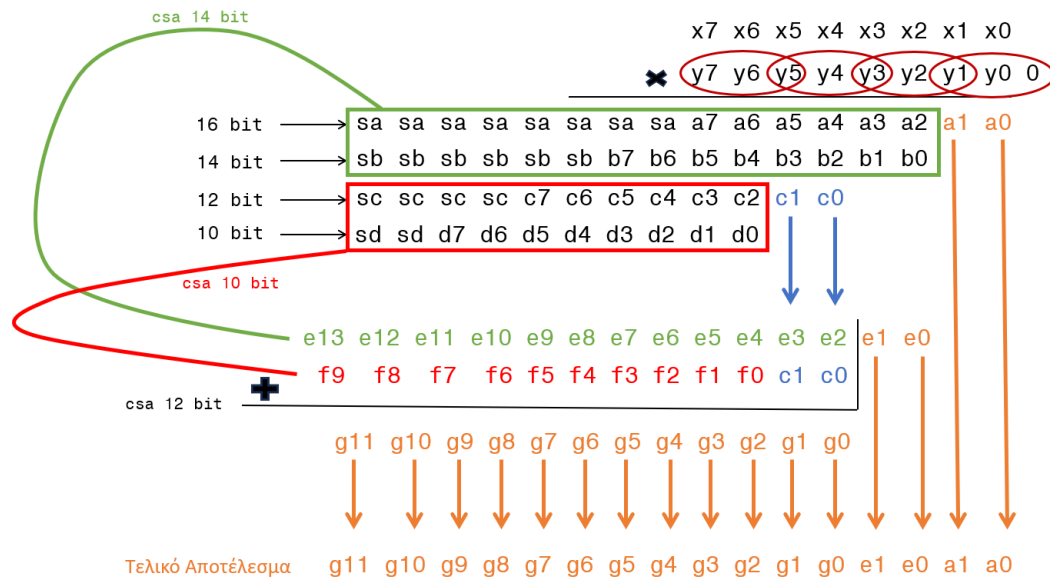
Τα τέσσερα μερικά γινόμενα που προκύπτουν με αυτή την διαδικασία γίνονται κατάλληλα sign extended μέχρι το 16^ο bit (για πολλαπλασιασμό 8-bit αριθμών) και στην συνέχεια προστίθενται μεταξύ τους για τον υπολογισμό του τελικού γινομένου. Το συνολικό δομικό διάγραμμα του πολλαπλασιαστή Booth φαίνεται στην ακόλουθη εικόνα.



Η υλοποίηση του αθροιστή των μερικών γινομένων γίνεται με στόχο την επίτευξη της μέγιστης δυνατής ταχύτητας. Στόχος είναι να προσθέσουμε όσο πιο γρήγορα γίνεται τους 4 όρους (16 bits, 14 bits, 12 bits, 10 bits) μετά τον Booth Encoder που φαίνονται στο Block Diagram. Προσπαθήσαμε να ομαδοποιήσουμε και να υπολογίσουμε παράλληλα όσο πιο πολλούς όρους μπορούμε και να μην λάβουμε υπόψη αχρείαστους όρους στις πράξεις (όπως οι a_1, a_2, e_1, e_2) που θα μας ανάγκαζαν να χρησιμοποιήσουμε έξτρα hardware και να έχουμε μεγαλύτερη καθυστέρηση. Για την επίτευξη της μέγιστης ταχύτητας χρησιμοποιήσαμε Carry Select αθροιστές για τις προσθέσεις των μερικών γινομένων. Εφόσον έχουμε έτοιμα τα μερικά γινόμενα με τον τρόπο που αναλύσαμε παραπάνω η διαδικασία που ακολουθείται στη συνέχεια είναι η εξής:

1. Τα LSB bits a_0 και a_1 περνάνε κατευθείαν στο τελικό αποτέλεσμα.
2. Προσθέτουμε σε έναν Carry Select Adder των 14 bits τους όρους $a[15:2]$ και $b[13:0]$ και παράλληλα σε έναν Carry Select Adder των 10 bits τους όρους $c[11:2]$ και $d[9:0]$. Από αυτές τις δύο προσθέσεις προκύπτουν οι όροι $e[13:0]$ και $f[9:0]$.
3. Τα bits e_0 και e_1 στο τελικό αποτέλεσμα και τα bits c_0 και c_1 περνάνε κατευθείαν στα LSB του όρου $f[9:0]$.
4. Προσθέτουμε σε έναν Carry Select Adder των 12 bits τους όρους $e[13:2]$ και $f[9:0] - c_1 - c_0$. Από αυτή την άθροιση προκύπτει ο όρος $g[11:0]$ που είναι οι υπολειπόμενοι όροι για το τελικό αποτέλεσμα.

Η διαδικασία αυτή φαίνεται αναλυτικά στην παρακάτω εικόνα.



Η υλοποίηση του πολλαπλασιαστή Booth που αναφέρθηκε προηγουμένως σε vhd1 περιγράφεται αναλυτικά στην συνέχεια της αναφοράς.

Κωδικοποιητής Booth

Ο κωδικοποιητής Booth υλοποιείται σύμφωνα με το διάγραμμα σε επίπεδο πυλών που παρουσιάστηκε παραπάνω με χρήση των εξής αρχείων:

- boothencoder.vhd για την υλοποίηση του 1-bit booth encoder που δημιουργεί τα select bits NEG, SINGLE και DOUBLE με βάση την τριάδα ψηφίων του πολλαπλασιαστέου.
- boothselector.vhd για την υλοποίηση του 1-bit booth selector, ο οποίος παίρνει ως είσοδο τα select bits του encoder και δύο διαδοχικά ψηφία του πολλαπλασιαστέου Y και παράγει ένα bit του μερικού γινομένου.
- boothcomplete.vhd και prgenerator.vhd για την υλοποίηση του συνολικού κωδικοποιητή Booth

boothencoder.vhd

```

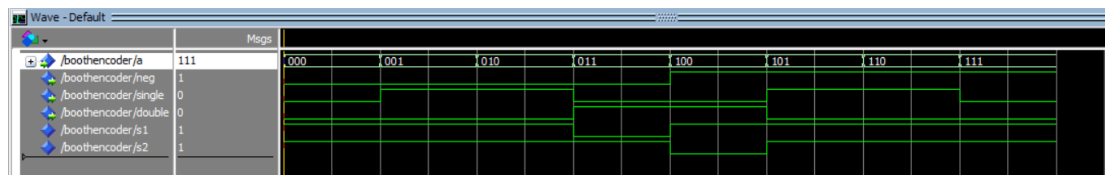
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_unsigned.all;

entity boothencoder is --kyklwma booth encoder me eisodo 3 bit
    port( --orismos i/o
        a: in std_logic_vector(2 downto 0);
        neg, single, double: out std_logic
    );
end boothencoder;

architecture arc of boothencoder is
    signal s1, s2: std_logic;
begin
    --ylopoihs symfwna me to schematic
    single <= a(0) xor a(1);
    s1 <= ((a(0) and a(1)) nand (not a(2)));
    s2 <= ((not a(0)) and (not a(1))) nand a(2);
    double <= s1 nand s2;
    neg <= a(2);
end arc;

```

Με προσομοίωσή του για όλες τις τιμές του πίνακα αληθείας μπορούμε να διαπιστώσουμε την σωστή λειτουργία του, όπως φαίνεται παρακάτω.



boothselector.vhd

```

library ieee;
use ieee.std_logic_1164.all;

entity boothselector is --kyklwma booth selector 1 bit
    port( --orismos i/o
        yn, ynl: in std_logic; --y(n) kai y(n-1)
        single, double, neg: in std_logic; --eisodoi elegxou apo ton booth decoder
        pp: out std_logic --partial product
    );
end boothselector;

architecture arc of boothselector is
    signal s1: std_logic;
begin
    --ylopoihs me vash to schematic
    s1 <= (ynl and single) nor (yn and double);
    pp <= (not s1) xor neg;
end arc;

```

boothcomplete.vhd

```

library ieee;
use ieee.std_logic_1164.all;

entity boothcomplete is
    port( --orismos i/o
        x: in std_logic_vector(2 downto 0); --3 bit gia encoder
        y: in std_logic_vector(7 downto 0); --pollaplasiasths
        pp: out std_logic_vector(7 downto 0) --partial product
    );
end boothcomplete;

architecture arc of boothcomplete is
    component boothselector --xrhsh component booth selector 1 bit
        port(
            yn, ynl: in std_logic; --y(n) kai y(n-1)
            single, double, neg: in std_logic; --eisodoi elegxou apo ton booth decoder
            pp: out std_logic --partial product
        );
    end component;

    component boothencoder --xrhsh component booth encoder me eisodo 3 bit
        port(
            a: in std_logic_vector(2 downto 0);
            neg, single, double: out std_logic
        );
    end component;

    component adder --xrhsh component adder
        generic (n :integer :=7);
        port(
            a,b:in std_logic_vector (n downto 0);
            cinn:in std_logic;
            s:out std_logic_vector (n downto 0);
            cout:out std_logic
        );
    end component;

    signal s, n, d: std_logic; --signals gia single, double, neg
    signal yin: std_logic_vector(8 downto 0);
    signal temp, sum, b: std_logic_vector(7 downto 0);
    signal cout: std_logic;
begin
    encoding: boothencoder port map(x, n, s, d);

    yin <= y & '0';
    gen_pp: for i in 0 to 7 generate
        U: boothselector port map(yin(i), yin(i+1), s, d, n, temp(i)); --pp se sumplhrwma ws pros 1
    end generate gen_pp;

    b <= "00000000";
    add: adder generic map(7) port map(temp, b, n, sum, cout); --pp se sumplhrwma ws pros 2

    pp <= sum;
end arc;

```

Αξίζει να αναφερθεί ότι η λογική της δομής του κωδικοποιητή Booth που έχει περιγραφεί μέχρι στιγμής παράγει το μερικό γινόμενο σε συμπλήρωμα ως προς 1 στην περίπτωση που αυτό είναι ίσο με -Y, γι' αυτό στο τέλος του κυκλώματος έχει προστεθεί ένας αθροιστής 8 bit που προσθέτει +1 στο μερικό γινόμενο για να παραχθεί το συμπλήρωμα ως προς 2. Η δομή του αθροιστή αυτού περιγράφεται παρακάτω.

Προσομοιώνοντας το κύκλωμα για διαφορετικές τιμές των εισόδων διαπιστώνουμε ότι λειτουργεί όπως είναι αναμενόμενο.

Wave - Default		Mips									
			000	001	010	011	100	101	110	111	
/boothcomplete/x	111										
/boothcomplete/y	00011001		00011001								
/boothcomplete/pp	00000000		00000000	00011001		00110010	11001110	11100111		00000000	
/boothcomplete/a	0										
/boothcomplete/b	1										
/boothcomplete/d	0										
/boothcomplete/yin	000110010		000110010								
/boothcomplete/temp	11111111		00000000	00011001		00110010	11001101	11100110		11111111	
/boothcomplete/sum	00000000		00000000	00011001		00110010	11001110	11100111		00000000	
/boothcomplete/b	00000000		00000000								
/boothcomplete/cout	1										

ppgenerator.vhd

```

library ieee;
use ieee.std_logic_1164.all;

entity ppgenerator is
    port( --orismos i/o
        x: in std_logic_vector(7 downto 0);
        y: in std_logic_vector(7 downto 0);
        a, b, c, d: out std_logic_vector(7 downto 0)
    );
end ppgenerator;

architecture arc of ppgenerator is
    component boothcomplete --xrhsh component ypologismoy partial product
        port(
            x: in std_logic_vector(2 downto 0); --3 bit gia encoder
            y: in std_logic_vector(7 downto 0); --pollaplasiasths
            pp: out std_logic_vector(7 downto 0) --partial product
        );
    end component;

    signal temp: std_logic_vector(8 downto 0);
    begin
        temp <= x & '0'; --x[-1]=0
        U1: boothcomplete port map(temp(2 downto 0), y, a); --ginetai 16 bits me sign extension
        U2: boothcomplete port map(temp(4 downto 2), y, b); --ginetai 14 bits me sign extension
        U3: boothcomplete port map(temp(6 downto 4), y, c); --ginetai 12 bits me sign extension
        U4: boothcomplete port map(temp(8 downto 6), y, d); --ginetai 10 bits me sign extension
    end arc;

```

Με προσομοίωσή του διαπιστώνουμε την ορθή λειτουργία του.

	Msgs	
/ppgenerator/x	00100111	00100111
/ppgenerator/y	00011001	00011001
/ppgenerator/a	11100111	11100111
/ppgenerator/b	00110010	00110010
/ppgenerator/c	11001110	11001110
/ppgenerator/d	00011001	00011001
/ppgenerator/temp	001001110	001001110

Άθροιση μερικών γινομένων

Για την άθροιση των μερικών γινομένων όπως αναφέρθηκε νωρίτερα χρησιμοποιείται Carry Select Adder των 10, 12 και 14 bits. Για την υλοποίηση αυτών των αθροιστών χρησιμοποιήθηκαν τα εξής αρχεία VHDL:

- fulladder.vhd και adder.vhd για την υλοποίηση αθροιστή 1-bit και n-bits αντίστοιχα
- mux2to1.vhd και muxnbit.vhd για την υλοποίηση πολυπλέκτη από 2 σε 1 του 1-bit και n-bits αντίστοιχα.
- csa_gen10and14bit.vhd για την υλοποίηση generic Carry Select Adder των 10 και 14 bits με ομάδες μήκους των 2 bits.
- csa_gen12bit.vhd για την υλοποίηση generic Carry Select Adder των 12 bits με ομάδες μήκους των 4 bits.
- partialproductadder.vhd για την υλοποίηση του τελικού αθροίσματος των μερικών γινομένων χρησιμοποιώντας τους carry save αθροιστές που αναλύσαμε.

Ακολουθεί για κάθε αρχείο ο κώδικας και η προσομοίωσή για ενδεικτικές τιμές ώστε να διαπιστωθεί η σωστή λειτουργία τους.

fulladder.vhd:

adder.vhd:

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.numeric_std.all;
4
5  entity adder is --orizw eisodous
6      generic (n :integer :=7);
7      port(a,b:in std_logic_vector (n downto 0);
8           cinn:in std_logic;
9           s:out std_logic_vector (n downto 0);
10          cout:out std_logic
11      );
12  end adder;
13
14  architecture my_arch of adder is
15      --kalw component full addder
16      component fulladder
17          port( a,b,cin:in std_logic;
18              s,cout: out std_logic
19          );
20      end component;
21
22      signal c: std_logic_vector(n downto 0);
23
24      begin
25          --ftiaxnw to prwto full addder kai meta me for generate tous upoloipous
26          U1: fulladder port map (a(0),b(0),cinn,s(0),c(0));
27          generate_label:
28              for i in 1 to n generate
29                  fulladder_i: fulladder port map (a(i), b(i), c(i-1),s(i),c(i));
30              end generate;
31          cout<=c(n);--vazw to cout
32
33      end my_arch;
34

```

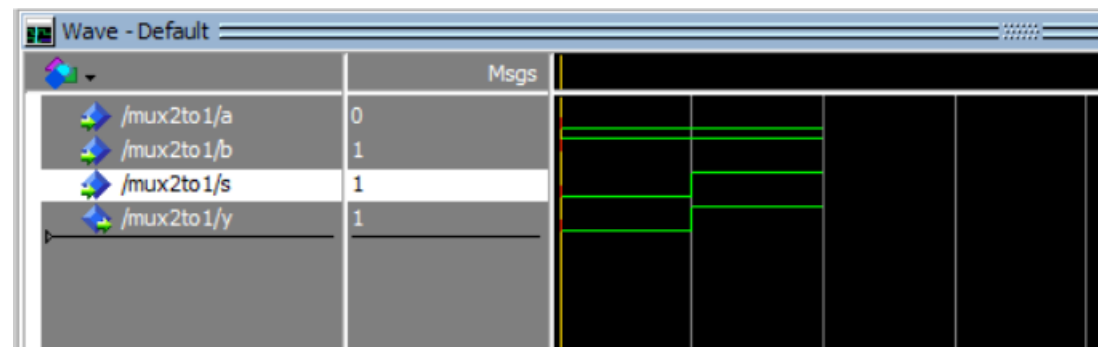
Wave - Default			
	Msgs		
/adder/a	11111111	11111111	
/adder/b	11111111	11111111	
/adder/cinn	0		
/adder/s	11111110	11111111	11111110
/adder/cout	1		
/adder/c	11111111	11111111	

mux2to1.vhd:

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.numeric_std.all;
4
5  entity mux2to1 is
6      port(a,b:in std_logic;--orizw shmata eisodou
7            s:in std_logic;
8            y:out std_logic
9          );
10 end mux2to1;
11
12 architecture my_arch of mux2to1 is
13     signal c1: std_logic; --eswtrika shmata gia na vohthisoun th logikh
14     signal c2: std_logic;
15 begin-- ylopoiwn thn eksiswsh tou mux  $y=(a*s')+(b*s)$ 
16     c1<=a and (not s);
17     c2<=b and s;
18     y<=c1 or c2;
19 end my_arch;
20

```



muxnbit.vhd:

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.numeric_std.all;
4
5  entity muxnbit is
6      generic (n: integer:=4); --kanw n-1 bit eisodous
7      port(a,b:in std_logic_vector (n-1 downto 0);--orizw shmata eisodou
8            s:in std_logic;
9            y:out std_logic_vector (n-1 downto 0)
10          );
11 end muxnbit;
12
13 architecture spuro of muxnbit is
14
15     component mux2to1--kalo san component to mux 2 se 1 enos bit
16     port ( a: in std_logic;
17            b: in std_logic;
18            s: in std_logic;
19            y: out std_logic);
20     end component;
21
22     begin --ftiaxnw n-1 muxes sthn ousia
23     gen_muxes: for i in 0 to n-1 generate
24         MUX_INST: mux2to1 port map (
25             a=> a(i),
26             b=> b(i),
27             s=> s,
28             y=> y(i)
29         );
30     end generate gen_muxes;
31 end spuro;
32

```

Wave - Default		
	Msgs	
/muxnbit/a	0001	0001
/muxnbit/b	1100	1100
/muxnbit/s	1	
/muxnbit/y	1100	0001 1100

csa_gen10and14bit.vhd:

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.numeric_std.all;
4
5  --auto pou allazw einai ton adder na ton kanw 2bito kai ta mux epishs
6
7  entity csa_gen10and14bit is
8      generic (n : integer := 13);
9      port(a,b:in std_logic_vector (n downto 0);--orizw shmata eisodou
10          cin:in std_logic;
11          s:out std_logic_vector (n downto 0);
12          cout:out std_logic
13          );
14  end csa_gen10and14bit;
15
16
17  architecture spuro of csa_gen10and14bit is
18
19      --kalo san component to mux 2 se 1 tou enos bit, to mux 2 se 1 tw n bits
20      --kai ton adder gia n=3 dld 4 bit etsi opws ton exw ulopoihsei
21      component mux2to1
22      port ( a: in std_logic;
23            b: in std_logic;
24            s: in std_logic;
25            y: out std_logic);
26      end component;
27
28      component muxnbit
29      generic (n: integer:=4);
30      port(a,b:in std_logic_vector (n-1 downto 0);
31          s:in std_logic;
32          y:out std_logic_vector (n-1 downto 0));
33      end component;
34
35      component adder
36      generic (n : integer :=7);
37      port(a,b:in std_logic_vector (n downto 0);
38          cinn:in std_logic;
39          s:out std_logic_vector (n downto 0);
40          cout:out std_logic
41          );
42      end component;

```

```

34
35 component adder
36   generic (n : integer := 7);
37   port(a,b:in std_logic_vector (n downto 0);
38       cin:in std_logic;
39       s:out std_logic_vector (n downto 0);
40       cout:out std_logic
41       );
42 end component;
43
44 --eswterika shmata gia na me vohthisoun sth logikh
45 signal s0: std_logic_vector (n downto 2); --sums twv athristwn pou perimenoun to carry
46 signal s1: std_logic_vector (n downto 2);
47
48 signal c: std_logic_vector (n/2 downto 0); --ta carry pou proxwraue sta muc
49 signal c0: std_logic_vector (n/2 downto 1); --ta carry pou vgainoun apo toys adder4 me carry in 0
50 signal c1: std_logic_vector (n/2 downto 1); --ta carry pou vgainoun apo toys adder4 me carry in 1
51
52
53
54 begin
55
56
57   U1: adder generic map (n=>1) port map (a(1 downto 0),b(1 downto 0),cin,s(1 downto 0),c(0));
58   --akolouthw th logikh apo to arxeio csagen12bit
59   generate_label:
60   for i in 1 to n/2 generate
61     adder_i0: adder generic map (n=>1) port map (a(2*i+1 downto 2*i),b(2*i+1 downto 2*i),'0',s0(2*i+1 downto 2*i),c0(i));
62     adder_i1: adder generic map (n=>1) port map (a(2*i+1 downto 2*i),b(2*i+1 downto 2*i),'1',s1(2*i+1 downto 2*i),c1(i));
63     mux2bit_i: muxnbit generic map (n=>2) port map (s0(2*i+1 downto 2*i),s1(2*i+1 downto 2*i),c(i-1),s(2*i+1 downto 2*i));
64     mux2tol_i: mux2tol port map (c0(i),c1(i),c(i-1),c(i));
65   end generate;
66   cout<= c(n/2);
67
68 end spuro;
69
70

```

Wave - Default		Msgs	
/csa_gen10and14bit/n	9	9	
/csa_gen10and14bit/a	0011001010	1111111111	0000000000 0011001010
/csa_gen10and14bit/b	0100000000	1111111111	0000000000 0100000000
/csa_gen10and14bit/s	0111001010	1111111111	0000000001 0111001010
/csa_gen10and14bit/c	0	10101010	00000000 01110010
/csa_gen10and14bit/c0	01110010	11111111	01010101 10000111
/csa_gen10and14bit/c1	10000111	111111	000000 0100
/csa_gen10and14bit/c2	000000	1111	0000
/csa_gen10and14bit/c3	0000	1111	0000
/csa_gen10and14bit/c4	0100	1111	0000

Wave - Default		Msgs	
/csa_gen10and14bit/n	13	13	
/csa_gen10and14bit/a	00010101100111	11111111111111	00000000000000 00010101100111
/csa_gen10and14bit/b	01000000000000	11111111111111	00000000000000 01000000000000
/csa_gen10and14bit/s	01010101100111	11111111111111	00000000000001 01010101100111
/csa_gen10and14bit/c	0	101010101010	000000000000 010101011001
/csa_gen10and14bit/c0	010101011001	11111111	0101010101 10101010110
/csa_gen10and14bit/c1	10101010110	11111111	00000000 00000000
/csa_gen10and14bit/c2	00000000	11111111	00000000
/csa_gen10and14bit/c3	00000000	11111111	00000000
/csa_gen10and14bit/c4	00000000	11111111	00000000

csa_gen12bit.vhd:

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.numeric_std.all;
4
5  --csa me block twm 4 fulladder
6
7  entity csa_gen12bit is
8      generic (n : integer := 11);
9      port(a,b:in std_logic_vector (n downto 0);--orizw shmata eisodou
10         cin:in std_logic;
11         s:out std_logic_vector (n downto 0);
12         cout:out std_logic
13         );
14  end csa_gen12bit;
15
16
17  architecture spuro of csa_gen12bit is
18
19      --kalo san component to mux 2 se 1 tou enos bit, to mux 2 se 1 twm n bits
20      --kai ton adder gia n=3 dld 4 bit etsi opws ton exw ulopoihsei
21      component mux2to1
22      port ( a: in std_logic;
23            b: in std_logic;
24            s: in std_logic;
25            y: out std_logic);
26      end component;
27
28      component muxnbit
29      generic (n: integer:=4);
30      port(a,b:in std_logic_vector (n-1 downto 0);
31          s:in std_logic;
32          y:out std_logic_vector (n-1 downto 0));
33      end component;
34
35      component adder
36      generic (n : integer :=7);
37      port(a,b:in std_logic_vector (n downto 0);
38          cinn:in std_logic;
39          s:out std_logic_vector (n downto 0);
40          cout:out std_logic
41          );
42      end component;
43
44
45      component adder
46      generic (n : integer :=7);
47      port(a,b:in std_logic_vector (n downto 0);
48          cinn:in std_logic;
49          s:out std_logic_vector (n downto 0);
50          cout:out std_logic
51          );
52      end component;
53
54      --eswterika shmata gia na me vohthisoun sth logikh
55      signal s0: std_logic_vector (n downto 4); --sums twm athristwn pou perimenoun to carry
56      signal s1: std_logic_vector (n downto 4);
57
58      signal c: std_logic_vector (n/4 downto 0);--ta carry pou proxwrane sta muc
59      signal c0: std_logic_vector (n/4 downto 1);--ta carry pou vgainoun apo toys adder4 me carry in 0
60      signal c1: std_logic_vector (n/4 downto 1);--ta carry pou vgainoun apo toys adder4 me carry in 1
61
62  begin
63
64      U1: adder generic map (n=>3) port map (a(3 downto 0),b(3 downto 0),cin,s(3 downto 0),c(0));
65      --akolouthw th logikh apo to arxelo csa.vhd sto lab3 askhsh apla to kanw me for generate
66      --ftiaxnw generic mhkous csa
67      generate_label:
68      for i in 1 to n/4 generate
69          adder_i0: adder generic map (n=>3) port map (a(4*i+3 downto 4*i),b(4*i+3 downto 4*i),'0',s0(4*i+3 downto 4*i),c0(i));
70          adder_i1: adder generic map (n=>3) port map (a(4*i+3 downto 4*i),b(4*i+3 downto 4*i),'1',s1(4*i+3 downto 4*i),c1(i));
71          mux4bit_i: muxnbit port map (s0(4*i+3 downto 4*i),s1(4*i+3 downto 4*i),c(i-1),s(4*i+3 downto 4*i));
72          mux2to1_i: mux2to1 port map (c0(i),c1(i),c(i-1),c(i));
73      end generate;
74      cout<=c(n/4);
75  end spuro;
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99

```

Wave - Default		Msgs																			
		Msgs																			
/csa_gen12bit/n		11		11																	
+ /csa_gen12bit/a		000101100111		111111111111		000000000000		000101100111													
+ /csa_gen12bit/b		010000000000		111111111111		000000000000		010000000000													
/csa_gen12bit/cin		0																			
+ /csa_gen12bit/s		010101100111		111111111111		000000000001		010101100111													
/csa_gen12bit/cout		0																			
+ /csa_gen12bit/s0		01010110		11101110		00000000		01010110													
+ /csa_gen12bit/s1		01100111		11111111		00010001		01100111													
+ /csa_gen12bit/c		000		111		000															
+ /csa_gen12bit/c0		00		11		00															
+ /csa_gen12bit/c1		00		11		00															

partialproductadder.vhd:

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.numeric_std.all;
4
5  --auto pou allazw einai ton adder na ton kanw 2bito kai ta mux epishs
6
7  entity partialproductadder is
8      port(a:in std_logic_vector (15 downto 0);--orizw shmata eisodou
9            b:in std_logic_vector (13 downto 0);
10           c:in std_logic_vector (11 downto 0);
11           d:in std_logic_vector (9 downto 0);
12           s:out std_logic_vector (15 downto 0)
13       );
14  end partialproductadder;
15
16  architecture spuro of partialproductadder is
17      --kalw ta components twm carry select adders
18
19      component csa_gen10and14bit
20          generic (n :integer :=13);
21          port(a,b:in std_logic_vector (n downto 0);
22                cin:in std_logic;
23                s:out std_logic_vector (n downto 0);
24                cout:out std_logic
25          );
26      end component;
27
28      component csa_gen12bit
29          generic (n :integer :=11);
30          port(a,b:in std_logic_vector (n downto 0);
31                cin:in std_logic;
32                s:out std_logic_vector (n downto 0);
33                cout:out std_logic
34          );
35      end component;
36
37      --eswterika shmata gia na perasw apo ta prwta duo csa ta apotelesmata sto 3o csa
38      signal s0: std_logic_vector (13 downto 0);
39      signal s1: std_logic_vector (11 downto 0);
40
41
42      begin --ulopoiw to block diagram ths anaforas
43          s(0)<=a(0);
44          s(1)<=a(1);

```

```

28 component csa_gen12bit
29     generic (n : integer := 11);
30     port(a,b:in std_logic_vector (n downto 0);
31         cin:in std_logic;
32         s:out std_logic_vector (n downto 0);
33         cout:out std_logic
34     );
35 end component;
36
37 --eswterika shmata gia na perasw apo ta prwta duo csa ta apotelesmata sto 3o csa
38 signal s0: std_logic_vector (13 downto 0);
39 signal s1: std_logic_vector (11 downto 0);
40
41
42 begin --ulopoiw to block diagram ths anaforas
43     s(0)<=a(0);
44     s(1)<=a(1);
45
46     --agnow ta carry eksodou twv csa opws ekshgeite sthn anafora giati xrhsimopoiw sumplhrwma ws pros 2 kai exw eksodo 16 bit pou einai to
47     --megisto egkuro apotelesma pou vgazei o booth multiplier mou
48     U1:csa_gen10and14bit generic map (n=>13) port map (a(15 downto 2),b(13 downto 0),'0',s0(13 downto 0),open);
49     U2:csa_gen10and14bit generic map (n=>9) port map (c(11 downto 2),d(9 downto 0),'0',s1(11 downto 2),open);
50     --vgazw ta duo lsb tou prwtou athroismatos sthn eksodo
51     s(2)<=s0(0);
52     s(3)<=s0(1);
53     --vazw ta duo lsb tou tritou merikou ginomenou sto eswtriko shma gia to teliko csa
54     s1(0)<=s(2);
55     s1(1)<=s(3);
56     --kanw ton teliko csa pou tha vgalei to apotelesma
57     V1:csa_gen12bit generic map (n=>11) port map (s0(13 downto 2),s1(11 downto 0),'0',s(15 downto 4),open);
58
59 end spuro;

```

Wave - Default		Msgs	
/partialproductadder/a	No Data-	0111111111111111	0100000000000000
/partialproductadder/b	No Data-	0111111111111111	0000000000011111
/partialproductadder/c	No Data-	0111111111111111	0000000010000
/partialproductadder/d	No Data-	0111111111111111	00110000000
/partialproductadder/s	No Data-	0111111110101011	0111000100111100
/partialproductadder/s0	No Data-	0111111111111110	0100000000011111
/partialproductadder/s1	No Data-	01111111111011	001100010000

Συνδυάζοντας τα κυκλώματα δημιουργίας και άθροισης των μερικών γινομένων, προκύπτει ο τελικός πολλαπλασιαστής Booth (αρχείο boothmultiplier.vhd), όπως φαίνεται παρακάτω.

```

library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity boothmultiplier is
    port(
        x, y: in std_logic_vector(7 downto 0); --pollaplasiasteoi
        s: out std_logic_vector(15 downto 0) --ginomeno
    );
end boothmultiplier;

architecture arc of boothmultiplier is
    component ppgenerator --xrhsh component partial product generator
        port(
            x: in std_logic_vector(7 downto 0);
            y: in std_logic_vector(7 downto 0);
            a, b, c, d: out std_logic_vector(7 downto 0)
        );
    end component;

    component partialproductadder --xrhsh component partial product adder
        port(
            a: in std_logic_vector(15 downto 0);
            b: in std_logic_vector(13 downto 0);
            c: in std_logic_vector(11 downto 0);
            d: in std_logic_vector(9 downto 0);
            s: out std_logic_vector(15 downto 0)
        );
    end component;

    signal s1, s2, s3, s4: std_logic_vector(7 downto 0);
    signal a: std_logic_vector(15 downto 0);
    signal b: std_logic_vector(13 downto 0);
    signal c: std_logic_vector(11 downto 0);
    signal d: std_logic_vector(9 downto 0);
    signal temp: std_logic_vector(15 downto 0);
    begin
        U1: ppgenerator port map(x, y, s1, s2, s3, s4);
        a <= s1(7) & s1(7) & s1(7) & s1(7) & s1(7) & s1(7) & s1(7) & s1(7);
        b <= s2(7) & s2(7) & s2(7) & s2(7) & s2(7) & s2(7) & s2(7);
        c <= s3(7) & s3(7) & s3(7) & s3(7) & s3(7);
        d <= s4(7) & s4(7) & s4(7);
        U2: partialproductadder port map(a, b, c, d, temp);

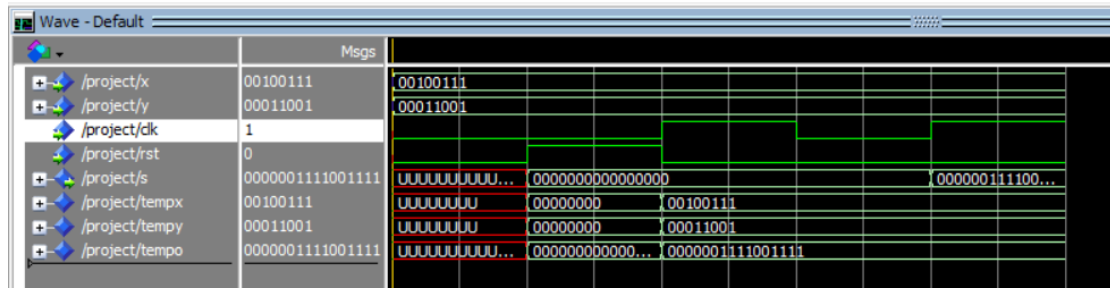
        s <= temp;
    end arc;

```

Τα μερικά γινόμενα που παράγονται από τον ppgenerator αθροίζονται στον partial product adder, αφού του γίνει κατάλληλη επέκταση προσήμου στο καθένα από αυτά.

Στο κύκλωμα αυτό προστίθενται επιπλέον καταχωρητές (QD.vhd) στην είσοδο και στην έξοδό του και παράγεται το τελικό κύκλωμα του πολλαπλασιαστή Booth (αρχείο project.vhd)

Με προσομοίωσή του μπορούμε να διαπιστώσουμε ότι λειτουργεί όπως είναι αναμενόμενο.



Στην συνέχεια γίνεται σύνθεση του τελικού κυκλώματος με χρήση genus για τεχνολογία 45 και 7 nm.

Για 45 nm:

Θέτοντας ρολόι με περίοδο 5 ns, παρατηρούμε ότι το κύκλωμα έχει critical path ίσο με 4.861 ns και 13 ps slack.

	Capture	Launch
Clock Edge:+	5000	0
Src Latency:+	0	0
Net Latency:+	0 (I)	0 (I)
Arrival:=	5000	0
Setup:-	126	
Required Time:=	4874	
Launch Clock:-	0	
Data Path:-	4861	
Slack:=	13	

#	Timing Point	Flags	Arc	Edge	Cell	Fanout	Load (fF)	Trans (ps)	Delay (ps)	Arrival (ps)	Instance	Location
#	U1_q_reg[1]/CK	-	-	R	(arrival)	32	-	0	0	0	(-, -)	
	U1_q_reg[1]/Q	-	CK->Q	F	DFFRX4	10	28.9	141	316	316	(-, -)	
	g3676_3759/Y	-	A->Y	R	NOR2X1	1	4.7	167	138	454	(-, -)	
	g3744/Y	-	A1N->Y	R	OAI2BB1X4	7	20.8	127	216	670	(-, -)	
	g3614_8428/Y	-	A1->Y	F	AOI21X1	1	5.0	208	161	831	(-, -)	
	g3601_1705/Y	-	B->Y	R	XNOR2X1	1	5.5	109	259	1090	(-, -)	
	g3592_5526/S	-	B->S	F	ADDHX1	1	5.2	110	217	1307	(-, -)	
	g3584_7410/S	-	A->S	R	ADDHX1	1	5.5	106	230	1537	(-, -)	
	g3578_7482/C0	-	B->C0	R	ADDHX1	1	4.5	93	166	1704	(-, -)	
	g3575_1881/Y	-	B->Y	R	OR2X1	1	5.5	95	122	1826	(-, -)	
	g3567_6783/C0	-	B->C0	R	ADDHX1	1	4.5	93	163	1989	(-, -)	
	g3561_4319/Y	-	B->Y	R	OR2X1	1	5.5	95	122	2111	(-, -)	
	g3550_4733/C0	-	B->C0	R	ADDHX1	1	4.5	93	163	2274	(-, -)	
	g3545_1881/Y	-	B->Y	R	OR2X1	1	5.5	95	122	2396	(-, -)	
	g3537_1617/C0	-	B->C0	R	ADDHX1	1	4.5	93	163	2559	(-, -)	
	g3534_6783/Y	-	B->Y	R	OR2X1	1	5.5	95	122	2681	(-, -)	
	g3531_6260/C0	-	B->C0	R	ADDHX1	1	4.5	93	163	2844	(-, -)	
	g3527_5107/Y	-	B->Y	R	OR2X1	1	5.5	95	122	2967	(-, -)	
	g3523_1666/C0	-	B->C0	R	ADDHX1	1	4.5	93	163	3130	(-, -)	
	g3522_2883/Y	-	B->Y	R	OR2X1	2	7.4	123	136	3266	(-, -)	
	g3518_9315/Y	-	B->Y	F	XNOR2X1	1	6.3	131	212	3478	(-, -)	
	g3506_7098/C0	-	B->C0	F	ADDFX1	4	13.0	253	321	3799	(-, -)	
	g3501_5122/Y	-	A2->Y	R	OAI31X1	2	7.2	390	283	4082	(-, -)	
	g3500_1617/Y	-	B->Y	R	AND2X2	4	13.3	132	304	4385	(-, -)	
	g3/Y	-	B->Y	F	MXI2X1	1	4.7	205	162	4547	(-, -)	
	g2/Y	-	A->Y	R	NAND2X1	1	5.1	125	124	4671	(-, -)	
	g3488_2398/Y	-	B->Y	R	CLKXOR2X1	1	4.7	99	190	4861	(-, -)	
#	U4_q_reg[13]/D <<<	-	-	R	DFFRHQX1	1	-	-	0	4861	(-, -)	

Επιπλέον, καταλαμβάνει χώρο ίσο με περίπου 947 cells.

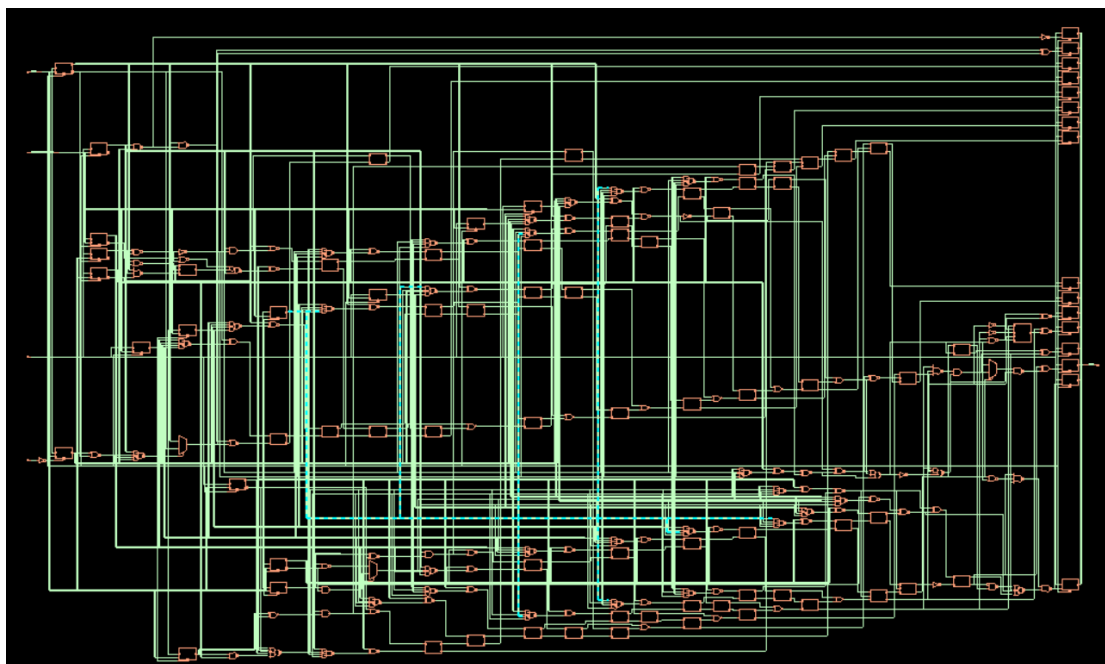
Instance	Module	Cell Count	Cell Area	Net Area	Total Area
project		208	669.294	277.657	946.951

Τέλος, φαίνεται πως καταναλώνει ισχύ ίση με 207.793 μW .

Instance: /project
Power Unit: W
PDB Frames: /stim#0/frame#0

Category	Leakage	Internal	Switching	Total	Row%
memory	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
register	9.99379e-09	8.97936e-05	1.40665e-05	1.03870e-04	49.99%
latch	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
logic	2.23371e-08	4.70808e-05	5.37252e-05	1.00828e-04	48.52%
bbox	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
clock	0.00000e+00	0.00000e+00	3.09420e-06	3.09420e-06	1.49%
pad	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
pm	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
Subtotal	3.23309e-08	1.36874e-04	7.08859e-05	2.07793e-04	100.00%
Percentage	0.02%	65.87%	34.11%	100.00%	100.00%

Το schematic του κυκλώματος όπως αυτό προκύπτει από το genus φαίνεται παρακάτω.



Για 7nm:

Με περίοδο ρολογιού ίση με 705 ps, το critical path ισούται με 684 ps και το κύκλωμα παρουσιάζει slack ίσο με 2 ps.

		Capture	Launch							
Clock Edge:+		705		0						
Src Latency:+		0		0						
Net Latency:+		0 (I)		0 (I)						
Arrival:=		705		0						
Setup:-		19								
Required Time:=		686								
Launch Clock:-		0								
Data Path:-		684								
Slack:=		2								
Timing Point	Flags	Arc	Edge	Cell	Fanout	Load (fF)	Trans (ps)	Delay (ps)	Arrival (ps)	Instance Location
U1_q_reg[0]/CLK	-	-	R	(arrival)	32	-	0	0	0	(-, -)
U1_q_reg[0]/QN	-	CLK->QN	F	DFFASRHQNx1_ASAP7_75t_R	1	0.6	20	50	50	(-, -)
U3_g5778/Y	-	A->Y	R	INVx1_ASAP7_75t_R	2	1.1	22	18	68	(-, -)
U3_g5777/Y	-	A->Y	F	INVx1_ASAP7_75t_R	8	3.8	41	29	97	(-, -)
U3_g5754_4319/Y	-	B->Y	R	NAND2xp5_ASAP7_75t_R	1	0.5	20	18	114	(-, -)
U3_g5707_7098/Y	-	A->Y	F	NAND2xp5_ASAP7_75t_R	1	1.0	18	14	129	(-, -)
U3_g5672_6783/Y	-	B->Y	F	XNOR2xp5_ASAP7_75t_R	2	2.0	38	32	160	(-, -)
U3_g5642_6783/Y	-	B->Y	R	NOR2x1_ASAP7_75t_R	2	1.4	24	20	180	(-, -)
U3_g5620_5115/Y	-	B->Y	F	NAND2xp5_ASAP7_75t_R	2	1.3	24	18	198	(-, -)
U3_g5602_5107/Y	-	B->Y	F	XNOR2xp5_ASAP7_75t_R	2	1.4	28	30	228	(-, -)
U3_g5584_6131/Y	-	B->Y	F	XOR2xp5_ASAP7_75t_R	2	1.5	30	28	256	(-, -)
U3_g5562_2346/Y	-	B->Y	R	NAND2xp5_ASAP7_75t_R	1	0.5	20	15	271	(-, -)
U3_g5552_5115/Y	-	B->Y	F	NAND2xp5_ASAP7_75t_R	2	1.5	24	18	290	(-, -)
U3_g5539_6783/Y	-	B->Y	R	NAND2xp5_ASAP7_75t_R	1	0.5	18	14	303	(-, -)
U3_g5526_2346/Y	-	B->Y	F	NAND2xp5_ASAP7_75t_R	2	1.5	23	18	321	(-, -)
U3_g5513_6131/Y	-	B->Y	R	NAND2xp5_ASAP7_75t_R	1	0.5	18	14	334	(-, -)
U3_g5504_1617/Y	-	B->Y	F	NAND2xp5_ASAP7_75t_R	2	1.4	22	17	351	(-, -)
U3_g5499_4319/Y	-	B->Y	R	NAND2xp5_ASAP7_75t_R	1	0.5	18	13	365	(-, -)
U3_g5493_2398/Y	-	B->Y	F	NAND2xp5_ASAP7_75t_R	2	1.4	22	17	382	(-, -)
U3_g5489_1666/Y	-	B->Y	R	NAND2xp5_ASAP7_75t_R	1	0.5	19	13	395	(-, -)
U3_g5484_9315/Y	-	B->Y	F	NAND2xp5_ASAP7_75t_R	3	1.6	25	18	413	(-, -)
U3_g5477_6131/Y	-	B->Y	R	NAND2xp33_ASAP7_75t_R	1	0.5	25	19	432	(-, -)
U3_g5468_6783/Y	-	B->Y	F	NAND2xp5_ASAP7_75t_R	1	0.9	18	16	448	(-, -)
U3_g5456_1666/Y	-	B->Y	F	XOR2xp5_ASAP7_75t_R	2	1.5	30	26	473	(-, -)
U3_g5444_6131/Y	-	B->Y	F	XNOR2xp5_ASAP7_75t_R	3	2.0	34	35	508	(-, -)
U3_g5436_3680/Y	-	B->Y	R	NAND2xp5_ASAP7_75t_R	1	0.5	17	16	524	(-, -)
U3_g5427_5107/Y	-	B->Y	F	NAND2xp5_ASAP7_75t_R	2	1.0	18	14	538	(-, -)
U3_g5424_6417/Y	-	B->Y	R	NAND2xp5_ASAP7_75t_R	1	0.5	18	12	551	(-, -)
U3_g5417_9315/Y	-	B->Y	F	NAND2xp5_ASAP7_75t_R	2	1.2	20	16	566	(-, -)
U3_g5413_7482/Y	-	B->Y	R	NAND2xp33_ASAP7_75t_R	1	0.5	26	17	584	(-, -)
U3_g5409_7098/Y	-	B->Y	F	NAND2xp5_ASAP7_75t_R	1	0.5	14	13	596	(-, -)
U3_g5408_8246/Y	-	B->Y	R	NAND2xp5_ASAP7_75t_R	1	0.5	20	11	608	(-, -)
U3_g5405_2802/Y	-	B->Y	R	AND2x2_ASAP7_75t_R	3	1.4	12	24	632	(-, -)
U3_g5404/Y	-	A->Y	F	INVxp67_ASAP7_75t_R	2	1.5	25	18	650	(-, -)
U3_g5402_3680/Y	-	B->Y	R	NAND2xp5_ASAP7_75t_R	1	0.5	15	14	664	(-, -)
g5800/Y	-	A->Y	R	AND2x2_ASAP7_75t_R	1	0.6	8	20	684	(-, -)
U4_q_reg[13]/D	<<<	-	R	DFFASRHQNx1_ASAP7_75t_R	1	-	-	0	684	(-, -)

Επίσης, καταλαμβάνει χώρο ίσο με 42.049 cells.

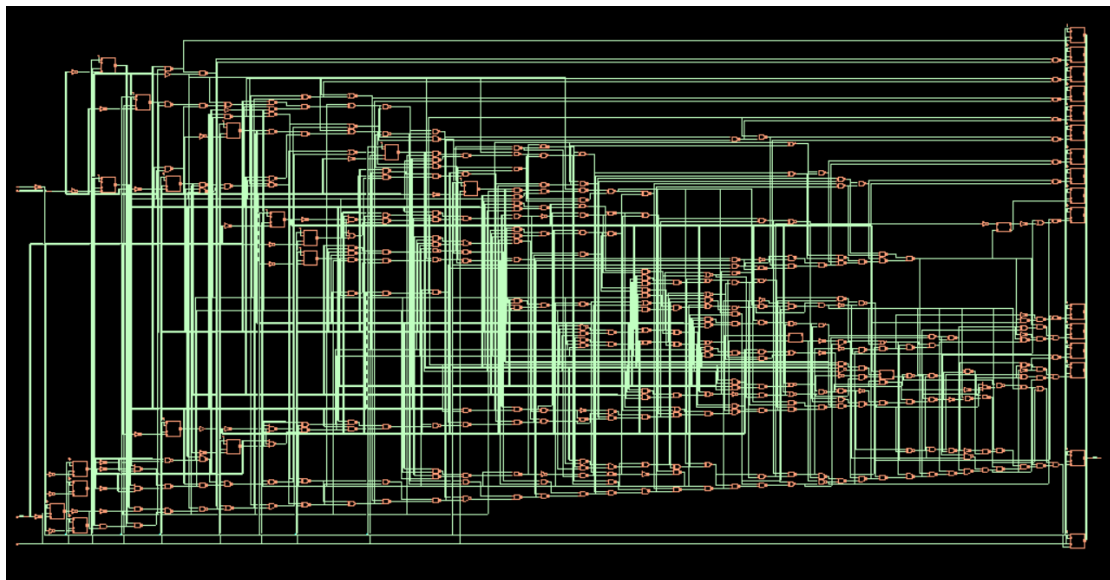
Instance	Module	Cell Count	Cell Area	Net Area	Total Area	Wireload
project		405	42.049	0.000	42.049	<none> (D)
(D) = wireload is default in technology library						

Τέλος, μπορούμε να διαπιστώσουμε ότι το κύκλωμα καταναλώνει ισχύ ίση με 211.143 μ W.

Instance: /project
Power Unit: W
PDB Frames: /stim#0/frame#0

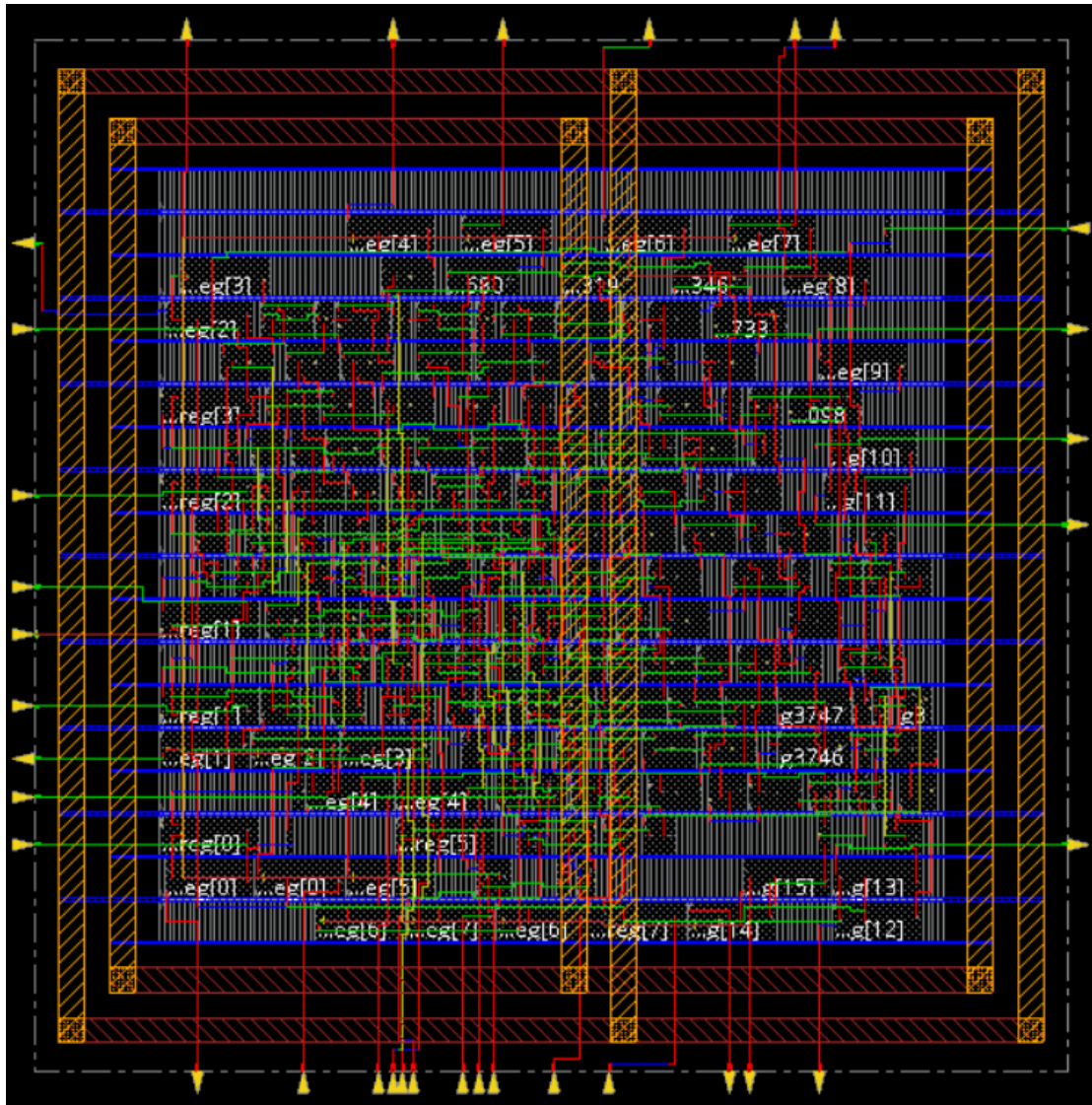
Category	Leakage	Internal	Switching	Total	Row%
memory	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
register	9.75158e-09	8.99705e-05	3.21801e-06	9.31983e-05	44.14%
latch	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
logic	6.15613e-08	4.66589e-05	6.01040e-05	1.06824e-04	50.59%
bbox	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
clock	0.00000e+00	0.00000e+00	1.11206e-05	1.11206e-05	5.27%
pad	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
pm	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
Subtotal	7.13129e-08	1.36629e-04	7.44426e-05	2.11143e-04	100.00%
Percentage	0.03%	64.71%	35.26%	100.00%	100.00%

Το schematic του κυκλώματος όπως προκύπτει από το genus φαίνεται στην ακόλουθη εικόνα.



Παρατηρούμε ότι στην περίπτωση των 7 nm, το κύκλωμα απαιτεί λιγότερο χώρο και ρολόι μικρότερης περιόδου για να λειτουργήσει, όπως είναι εξάλλου αναμενόμενο λόγω της τεχνολογίας. Επιπλέον, φαίνεται πως η κατανάλωση ισχύος των δύο κυκλωμάτων είναι περίπου ίση, το οποίο οφείλεται στην πολύ μεγαλύτερη συχνότητα στην περίπτωση των 7 nm (δεδομένου ότι η κατανάλωση ισχύος δίνεται προσεγγιστικά από την $P = aCV_{DD}f$, όπου f η συχνότητα).

Με βάση τα outputs του genus για την τεχνολογία των 45 nm, δημιουργείται το layout του κυκλώματος με χρήση του εργαλείου ipnnovus, όπως φαίνεται παρακάτω.



Για την υλοποίησή του, έχουν τοποθετηθεί τα πινάκια των εισόδων και εξόδων με βάση το optimal placement που πραγματοποιεί το εργαλείο και έχει χρησιμοποιηθεί clock tree synthesis.

Μετά προχωρήσαμε στην προσομοίωση στο XCELIUM με το testbench (tb.v) που φαίνεται παρακάτω για να επιβεβαιώσουμε την ορθή λειτουργία του κυκλώματος:

```

1  `timescale 1ns / 1ps
2
3  module tb_project;
4
5      // Dilwsi simatwn Testbench (8-bit eisodoi, 16-bit eksodos)
6      reg [7:0] x_tb;
7      reg [7:0] y_tb;
8      reg clk_tb;
9      reg rst_tb;
10
11     // Eksodos tou kyklwmatos
12     wire [15:0] s_tb;
13
14     // Diasyndesi me to Top-level Component (UUT)
15     project uut (
16         .x(x_tb),
17         .y(y_tb),
18         .clk(clk_tb),
19         .rst(rst_tb),
20         .s(s_tb)
21     );
22
23     // Dimiourgia rologiou (Pio argo roloi: Periodos T = 100ns)
24     initial begin
25         clk_tb = 1'b0;
26     end
27     always #50 clk_tb = ~clk_tb;
28
29     // Senaria dokimis (Stimulus)
30     initial begin
31
32
33         // 1. Katastasi RESET
34         // Ksekinaei sta 0ns. To roloi einai LOW.
35         rst_tb = 1'b1;

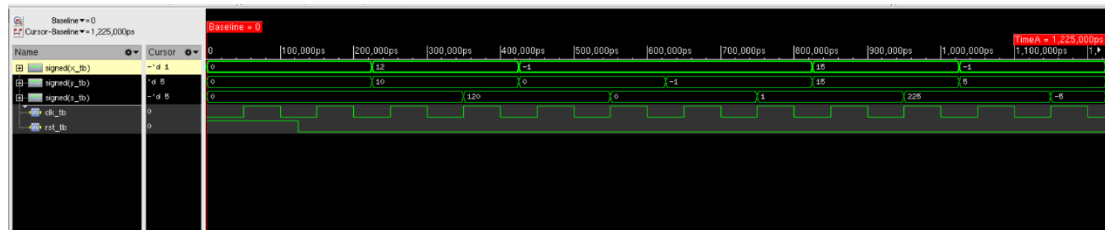
```

```

32
33 // 1. Katastasi RESET
34 // Ksekinaei sta 0ns. To roloi einai LOW.
35 rst_tb = 1'b1;
36 x_tb = 8'd0;
37 y_tb = 8'd0;
38
39 // Perimenoume 125ns. Ekeinh th stigmh to roloi einai LOW (0),
40 // opote apofeygoume ta setup violations.
41 #125;
42 rst_tb = 1'b0;
43
44 // 2. DOKIMI 1: Pollaplasiasmos thetikwn arithmwv
45 // Pame sta 225ns (to roloi einai pali LOW). Allazoume ta dedomena me asfaleia!
46 #100;
47 x_tb = 8'd12;
48 y_tb = 8'd10;
49
50 // Perimenoume 2 plireis kyklous rologiou (200ns) gia na ginei i praksi
51 #200;
52
53 // 3. DOKIMI 2: Pollaplasiasmos me to miden
54 x_tb = 8'd255;
55 y_tb = 8'd0;
56 #200;
57
58 // 4. DOKIMI 3: Pollaplasiasmos oriakwn timwn (Max unsigned values)
59 x_tb = 8'd255;
60 y_tb = 8'd255;
61 #200;
62
63 // 5. DOKIMI 4: Tyxaies times
64 x_tb = 8'd15;
65 y_tb = 8'd15;
66
67 // Perimenoume 2 plireis kyklous rologiou (200ns) gia na ginei i praksi
68 #200;
69
70 // 3. DOKIMI 2: Pollaplasiasmos me to miden
71 x_tb = 8'd255;
72 y_tb = 8'd0;
73 #200;
74
75 // 4. DOKIMI 3: Pollaplasiasmos oriakwn timwn (Max unsigned values)
76 x_tb = 8'd255;
77 y_tb = 8'd255;
78 #200;
79
80 // 5. DOKIMI 4: Tyxaies times
81 x_tb = 8'd15;
82 y_tb = 8'd15;
83 #200;
84
85 // 6. DOKIMI 5: Elegxos me arnitikous arithmous (Booth signed)
86 x_tb = 8'hFF;
87 y_tb = 8'd5;
88 #200;
89
90 $display("Booth Multiplier Simulation Finished.");
91 $stop;
92 end
93 endmodule

```

Στη συνέχεια ακολουθούν τα αποτελέσματα της προσομοίωσης (κάντε zoom για να τα δείτε):



Ακολουθεί η προσομοίωση στο LEC που επιβεβαιώνει ότι το κύκλωμα που δημιούργησε το genus (project_m.v) δουλεύει με τον ίδιο τρόπο και σωστά με το κύκλωμα vhd (project.vhd) που φτιάξαμε στο Modelsim.

Το αρχείο txt που υλοποιήσαμε:

```
1#LEC.tcl
2#start logic equivalence checker
3#lec -xl -nogui -color -64 #esvhsa auth thn entol;h kai thn evala sto terminal gt den douleue
4#read library
5read Lab5/fast_vddiv0_basicCells.W -verilog -both
6
7##read initial RTL design
8#read design Lab5/comparator.vhd Lab5/bitonic sorter.vhd Lab5/QD.vhd Lab5/askhsh.vhd -vhd -golden
9
10read design Project/adder.vhd Project/boothcomplete.vhd Project/QD.vhd Project/fulladder.vhd Project/csa_gen12bit.vhd Project/mux2to1.vhd Project/muxnbit.vhd Project/partialproductadder.vhd
Project/ppgenerator.vhd Project/csa_gen14bit.vhd Project/boothselector.vhd Project/boothmultiplier.vhd Project/boothencoder.vhd Project/project.vhd -vhd -golden
11
12
13##read netlist produced from genus
14read design Project/project_m.W -verilog -revised
15
16set system mode lec
17
18add compare point -all
19
20compare
21
22report verification
23
24exit
```

Τα αποτελέσματα του LEC που δίνουν PASS δηλαδή ότι τα δύο κυκλώματα δουλεύουν με τον ίδιο τρόπο:

```

=====
Mapped points: SYSTEM class
-----
Mapped points      PI      P0      DFF      Total
-----
Golden             18      16      32      66
-----
Revised            18      16      32      66
=====
// Command: add compare point -all
// 48 compared points added to compare list
// Command: compare
=====
Compared points      P0      DFF      Total
-----
Equivalent           16      32      48
=====
// Command: report verification
=====
                          Verification Report
-----
Category                                                    Count
-----
1. Non-standard modeling options used:                      0
-----
2. Incomplete verification:                                  0
-----
3. User modification to design:                              0
-----
4. Conformal Constraint Designer clock domain crossing checks recommended: 0
-----
5. Design ambiguity:                                         0
-----
6. Compare Results:                                          PASS
=====
// Command: exit
Do you really want to exit (Yes/No) ? █

```

Συμπερασματικά, στην συγκεκριμένη εργασία παρουσιάστηκε η δομή ενός radix-4 πολλαπλασιαστή Booth των 8 bit και υλοποιήθηκε σε επίπεδο hardware με χρήση vhdl. Πραγματοποιήθηκε σύνθεσή του μέσω genus, ελέγχθηκε η σωστή λειτουργία του με χρήση xcelium και lec και τελικά δημιουργήθηκε το layout του.

Πηγές

[1] N. Weste and D. Harris, “CMOS VLSI Design: A Circuits and Systems Perspective,” 3rd Edition, Pearson Addison Wesley, Boston, 2005.